

Meituan-Robotics-0: A Vision-Language-Action Foundation Model for Desktop Manipulation

Meituan Robotics Team

ABSTRACT

General-purpose desktop manipulation requires an embodied system that can acquire broad behavioral priors from heterogeneous data, transfer them across robot embodiments, recover from closed-loop execution errors, and preserve task-relevant state over extended horizons. We present Meituan-Robotics-0 (MR0), a vision-language-action foundation model and embodied-agent system for stationary-base desktop manipulation. Its data foundation combines 147 million vision-language samples with 15,060 hours of curated robot trajectories spanning 12 embodiments. MR0 converts this heterogeneous experience into executable control through a three-stage training recipe. Discrete action-token pre-training first aligns multimodal representations with robot behavior. Continuous flow-matching mid-training then transfers these priors to fine-grained cross-embodiment control. Real-world post-training further adapts the policy using successful demonstrations, human-gated corrective trajectories, and outcome-labeled autonomous rollouts. A unified end-effector interface supports learning across heterogeneous embodiments, while asynchronous action-chunk merging maintains smooth execution under variable inference latency. Because the low-level policy is reactive, an event-driven agent converts visual observations into persistent task events and invokes high-level planning only when task progress or feasibility changes. This separation preserves continuous motor control while enabling cross-task retrieval and conditional task resumption. Evaluated as a standalone policy, MR0 achieves 93.4% success on RoboTwin 2.0 and ranks fourth overall on RoboDojo, including first place in Open-vocabulary instruction following. Under matched HG-DAGger post-training across three real-robot tasks, it reaches a macro-average success rate of 0.78, compared with 0.69 for $\pi_{0.5}$. Applying RECAP to the precision-sensitive Ethernet-insertion task raises the final task-specific configuration to a macro-average success rate of 0.86 and a progress score of 0.93. Qualitative agent deployments further demonstrate cross-task event retrieval and recovery from temporarily unsatisfied task conditions. These results show that aligned heterogeneous data, closed-loop policy optimization, and event-level memory provide complementary foundations for transferable and long-horizon desktop manipulation.

1 Introduction

Vision-language-action [8, 46, 35, 39, 43, 34, 10] models provide a general interface for mapping visual observations and language instructions to robot actions. Their increasing data and model scales have substantially broadened the range of manipulation behaviors that can be learned within a single policy. Deploying such policies on desktop manipulation, however, exposes several limitations that cannot be addressed by scaling the action model alone. Robot datasets [1, 18, 41] collected from different sources use inconsistent observation formats, action spaces, embodiments, and task descriptions, which prevents them from providing coherent supervision without careful alignment. Even after data alignment, policies trained primarily on successful demonstrations receive little guidance for recovering from the error states induced by their own closed-loop behavior. Moreover, a reactive policy conditioned only on the current observation cannot preserve task information once it leaves the observation window. A practical desktop-manipulation system must therefore connect aligned heterogeneous data, robust closed-loop policy learning, and persistent task-level memory.

We present **Meituan-Robotics-0 (MR0)**, a desktop-manipulation framework built around a large-scale data foundation, a generalist VLA policy, and an event-driven embodied agent, as summarized in Figure 1. The data foundation combines 147 million vision-language samples with 15,060 hours of curated robot trajectories spanning 12 embodiments. The vision-language corpus provides general visual-semantic knowledge together with embodied supervision for grounding, scene understanding, affordance prediction, and task reasoning. The robot corpus comprises three complementary components: **RoboDesk**, a curated collection of open-source real-robot trajectories totaling 3,235 hours; 7,530 hours of

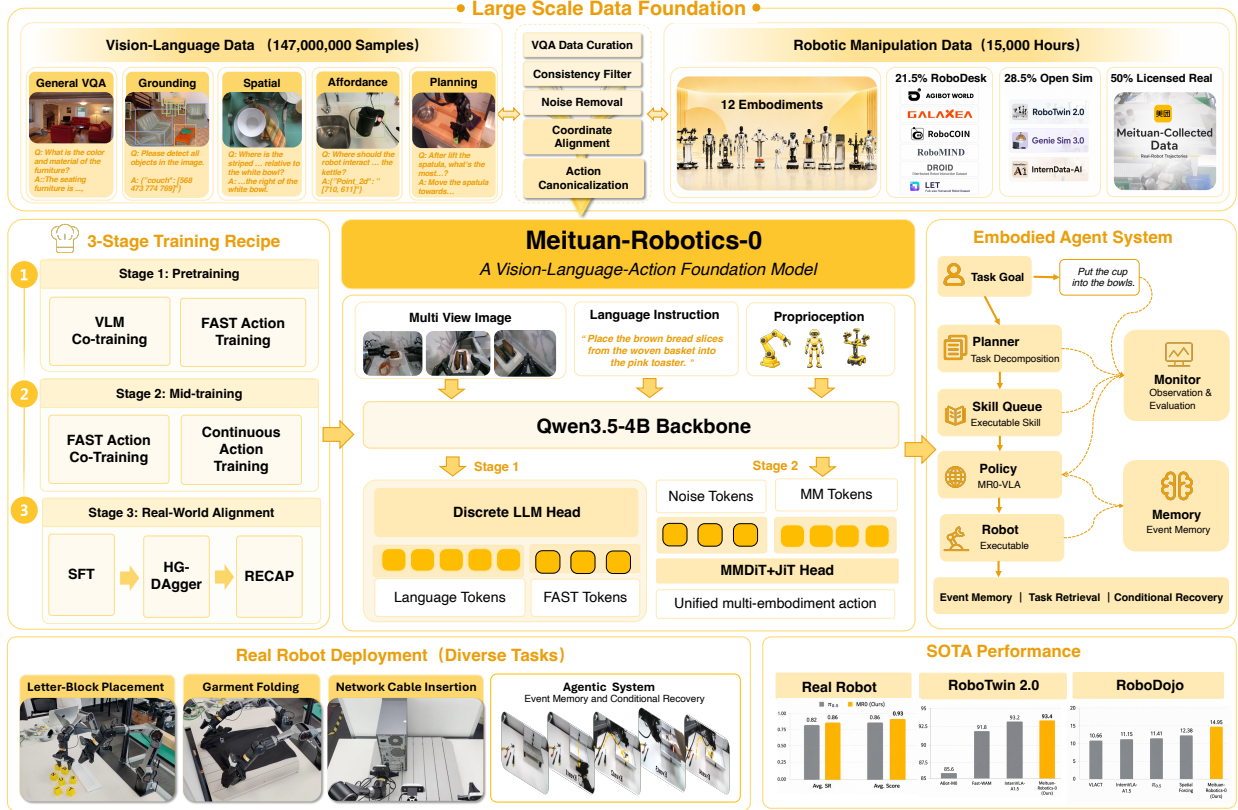


Figure 1: **Overview of Meituan-Robotics-0.** The framework integrates large-scale multimodal data, three-stage VLA training, an event-driven embodied-agent system, and real-robot deployment.

licensed real-robot data spanning seven robot configurations; and 4,295 hours of open simulation data. The complete corpus is processed through temporal and visual filtering, kinematic-consistency checks, cross-embodiment action canonicalization, and hierarchical task alignment. RoboDesk and the simulation data broaden task, environment, and embodiment coverage, while the licensed real-robot data provide task-focused manipulation experience.

Building on this aligned data foundation, the MRO policy follows a three-stage training recipe that separates the acquisition of broad action priors from continuous-control learning and real-world adaptation. A shared multimodal backbone is first paired with a discrete action head and pre-trained jointly on vision-language data and tokenized robot actions. Continuous mid-training then introduces an MMDiT [42] action head and optimizes flow matching over a unified end-effector representation, transferring the pre-trained features to fine-grained control across single-arm and bimanual embodiments. Real-world post-training [51, 22, 23] begins with supervised finetuning on successful demonstrations and subsequently considers two closed-loop optimization strategies. HG-Dagger [17] collects human corrections in policy-induced error states, whereas RECAP [37] additionally uses outcome-labeled autonomous rollouts to learn value- and advantage-conditioned behavior. During deployment, asynchronous action-chunk merging aligns newly predicted actions with the commands already being executed, maintaining smooth control without blocking the robot on policy inference.

Closed-loop policy optimization improves local recovery, but the resulting policy remains reactive and maintains no temporal history beyond its current observation. MRO therefore places an event-driven embodied agent [56, 25, 13] above the low-level policy to manage long-horizon task state. A Monitor summarizes short visual windows into task-relevant events, and a Memory Manager consolidates these events into a persistent record of task progress and environmental conditions. The Planner updates the active subtask only when a new event changes task progress, reveals a failure, or alters whether the current objective is feasible. Motor execution consequently remains with the VLA policy, while high-level reasoning is activated only when needed. This separation enables the system to retrieve information acquired in earlier tasks, suspend an objective whose prerequisites are temporarily unavailable, and resume it after the required conditions are restored.

We evaluate the policy and agent under protocols matched to their respective roles. As a standalone policy, MRO achieves 93.4% average success on RoboTwin 2.0 [31]. On RoboDojo [9], it ranks fourth overall and first in Open-vocabulary

Distribution of Vision-Language Data

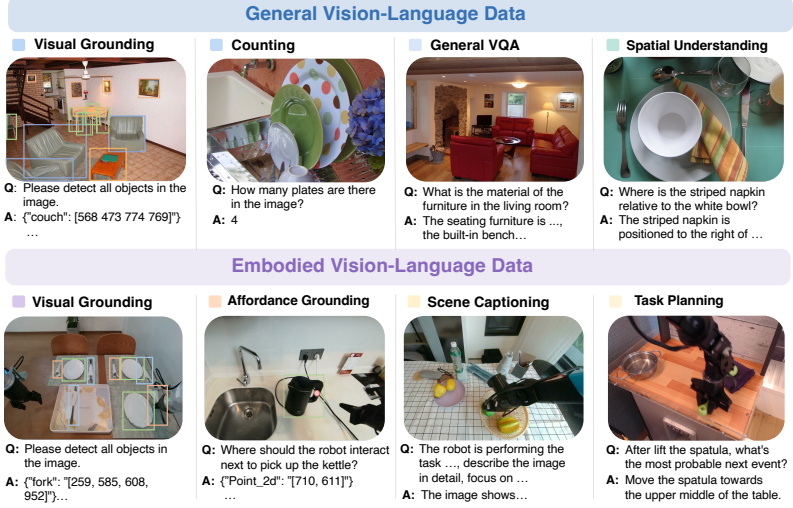
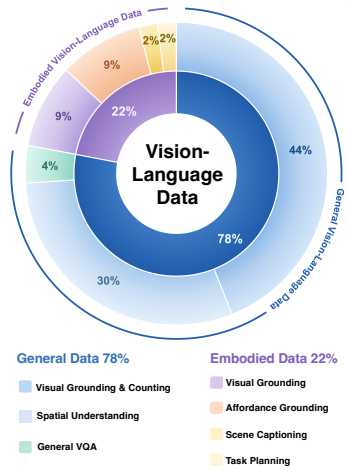


Figure 2: **Vision-language dataset.** The dataset contains 147 million samples, comprising 78% general vision-language data and 22% embodied vision-language data. Representative examples of the major data types are shown on the right.

instruction following, demonstrating strong transfer to diverse simulation tasks and underrepresented instructions. Under matched HG-DAGger post-training across three real-robot tasks, MR0 reaches a macro-average success rate of 0.78 and a progress score of 0.91, compared with 0.69 and 0.85 for $\pi_{0.5}$ [3]. Human-intervention data increase the MR0 success rate from 0.18 to 0.78, showing that supervision in policy-induced states is essential for closed-loop recovery. Applying RECAP to Ethernet insertion further raises the final task-specific configuration to a macro-average success rate of 0.86 and a progress score of 0.93. Separately, qualitative agent deployments demonstrate cross-task event retrieval and conditional task resumption. Together, these evaluations show how aligned data, continuous policy learning, closed-loop optimization, and event-level memory address complementary requirements of general-purpose desktop manipulation.

2 Data Curation

To equip MR0 with both broad visual-semantic understanding and diverse cross-embodiment manipulation experience, we independently curate two complementary data resources: 147 million vision-language samples and 15,060 hours of robot trajectories spanning 12 embodiments. The vision-language corpus provides supervision for general perception and embodied reasoning. Constructed through a separate pipeline, the robot corpus combines RoboDesk, licensed real-robot data, and open simulation data for action learning across heterogeneous hardware. We describe the two resources in Secs. 2.1 and 2.2, respectively.

2.1 Vision-Language Data

Visual reasoning for manipulation requires both broad perceptual knowledge and embodied understanding grounded in robot interaction. As shown in Figure 2, we organize the 147M vision-language corpus into two complementary subsets. General-purpose data account for 78% and provide supervision for visual question answering, spatial understanding, visual grounding, and counting. The remaining 22% consist of embodied data for scene captioning, task planning, visual grounding, and affordance grounding. Affordance grounding jointly localizes the manipulated object with a bounding box and identifies a 2D interaction point at which the robot should act.

For the embodied VQA component, we augment the collected open-source datasets with a large-scale in-house corpus derived from real-robot trajectories. To determine which supervision types should be prioritized in this corpus, we

Table 1: **Ablation of embodied vision-language supervision in the CALVIN ABC→D setting.** Bold indicates the highest Avg. Len. within each capability family.

Capability family	Training setting	Avg. Len.	Task 1	Task 2	Task 3	Task 4	Task 5
Baseline	—	3.832	91.0%	83.0%	76.2%	70.2%	62.8%
Spatial localization	+ 2D Grounding	3.941	91.3%	83.7%	78.1%	73.2%	67.8%
	+ 2D Pointing	3.806	90.8%	81.8%	74.8%	69.9%	63.3%
	+ 3D Grounding	4.037	92.4%	85.3%	79.6%	75.7%	70.7%
Action-related understanding	+ 2D Target-Affordance Grounding	4.102	93.3%	87.9%	82.5%	76.4%	70.1%
	+ 3D Target-Affordance Grounding	4.098	93.7%	89.9%	81.2%	73.9%	71.1%
	+ 2D Trajectory	3.987	92.0%	85.0%	79.2%	74.0%	68.5%
Semantic and task reasoning	+ Subtask Prediction	3.851	90.0%	82.2%	76.2%	71.7%	65.0%
	+ Scene Captioning	4.082	93.1%	87.1%	81.4%	76.2%	70.4%

conduct controlled CALVIN ABC-to-D ablations over eight candidate tasks and use the results to guide subsequent large-scale data construction. The candidates span three capability families: spatial localization, action-related understanding, and semantic and task reasoning. All variants use the same StarVLA [38] configuration, are trained on environments A, B, and C, and are evaluated on environment D. Table 1 reports success rates for completing sequences of one to five consecutive subtasks, together with the average completed sequence length. The results support including 2D target-affordance grounding and scene captioning as the action-related and semantic supervision types, respectively. For spatial localization, we choose 2D grounding because it avoids costly geometric annotations and scales more readily than 3D grounding. Taken together, the empirical results and practical annotation considerations lead us to focus our in-house data expansion on three types: 2D target-affordance grounding, scene captioning, and 2D grounding.

To ensure consistent and scalable data construction, we develop task-specific annotation pipelines for the selected supervision types. For 2D grounding, we apply Set-of-Mark prompting [48]: the Segment Anything Model (SAM) [19] proposes and marks candidate regions, and a vision-language model assigns an object identity to each region to form grounding question-answer pairs. For 2D affordance grounding, we obtain the target bounding box and interaction point from the key interaction frame of each action phase and propagate both annotations to earlier frames within that phase. For scene captioning, a vision-language model conditions on the robot observation and task context to describe task-relevant objects, spatial relations, gripper state, and execution progress.

2.2 Robotic Data

To provide diverse action supervision for robot control, we assemble 15,060 hours of tabletop trajectories spanning 12 embodiments. We restrict the corpus to stationary-base manipulation and exclude segments containing base motion. As summarized in Figure 3, the corpus consists of RoboDesk (3,235 hours; 21.5%), licensed real-robot data (7,530 hours; 50.0%), and open simulation data (4,295 hours; 28.5%). RoboDesk denotes the six open-source real-world datasets retained after curation, which are listed together with the simulation sources in Table 2.

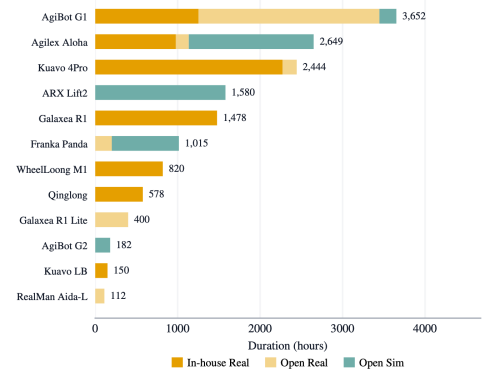
2.2.1 Data Curation Pipeline.

Raw trajectories contain idle boundaries, inconsistent kinematic records, visual defects, embodiment-specific action conventions, and uneven language annotations. We address these issues with three filtering stages (S1–S3) followed by two alignment stages (A1–A2). S1 removes static prefixes and suffixes, S2 rejects inconsistent joint–EEF recordings, and S3 removes corrupted visual segments. A1 canonicalizes actions across embodiments, while A2 aligns multi-granularity task annotations with the retained trajectories. Figure 4 summarizes the pipeline.

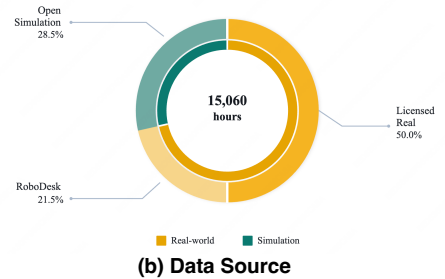
Static-boundary filtering. Teleoperated recordings often begin before manipulation and continue through task completion or reset, producing long intervals of near-zero motion at episode boundaries. Let $q_t \in \mathbb{R}^J$ be the recorded joint vector and $\Delta q_t^{(j)} = q_t^{(j)} - q_{t-1}^{(j)}$. A time index is considered static only when every joint remains below a motion threshold for L consecutive steps:

$$\text{Static}(t) \iff \left| \Delta q_{t+\ell}^{(j)} \right| < \tau_{\text{stat}}, \quad \forall j \in \{1, \dots, J\}, \quad \forall \ell \in \{0, \dots, L-1\}. \quad (1)$$

The persistence window prevents brief low-speed motions from being classified as idle. We trim only static runs connected to an episode boundary, preserving deliberate pauses during task execution. The corresponding video, state, and action intervals are removed jointly to maintain temporal synchronization.



(a) Embodiment Composition



(b) Data Source

Figure 3: Composition of the robot corpus. The 15,060-hour corpus spans 12 embodiments and comprises RoboDesk, a curated collection of open-source real-robot trajectories totaling 3,235 hours (21.5%); 7,530 hours (50.0%) of licensed real-robot data; and 4,295 hours (28.5%) of open simulation data.

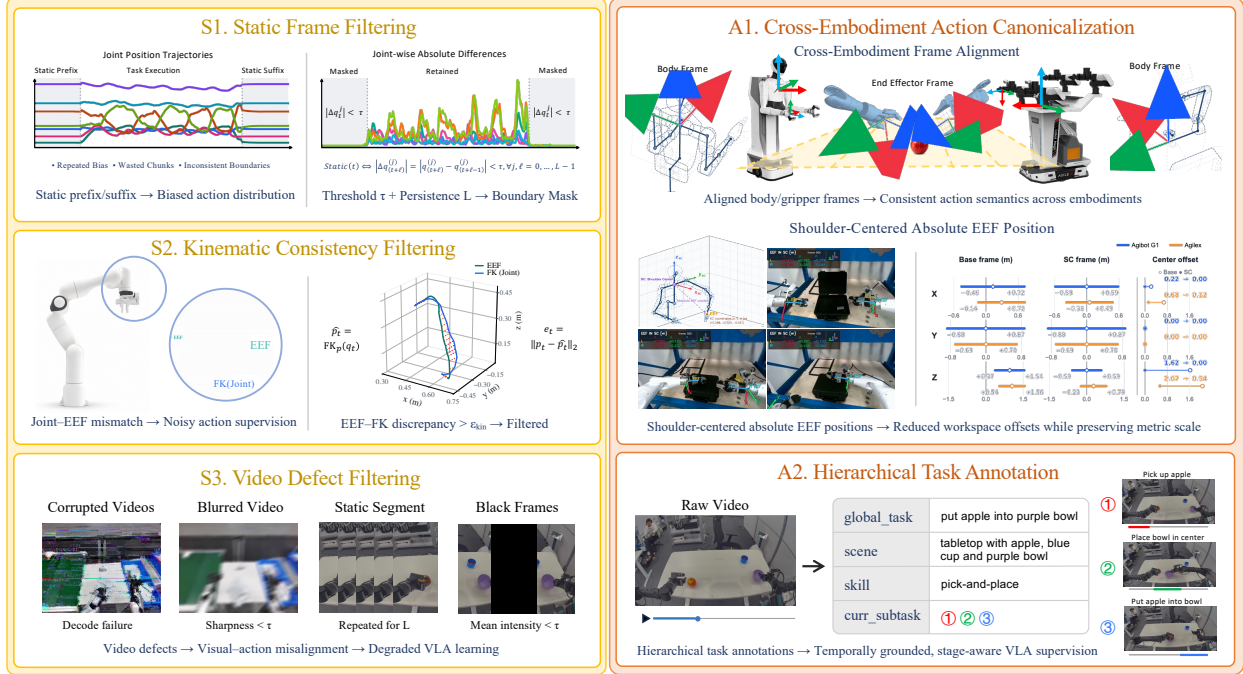


Figure 4: **Robot trajectory curation and alignment pipeline.** S1–S3 remove static boundaries, kinematic inconsistencies, and video defects; A1 canonicalizes actions across embodiments, and A2 aligns multi-granularity task annotations with the retained trajectories.

Kinematic-consistency filtering. For each embodiment, we reconstruct the end-effector position from the recorded joints using the corresponding URDF and compare it with the logged EEF position in a common frame. Samples exceeding an embodiment-specific error threshold are masked, and trajectories with persistent violations are discarded.

Video-defect filtering. We detect decoding failures, blur, frozen segments, and black frames. Corrupted intervals are removed from all synchronized streams, while episodes with persistent defects are discarded. Detection thresholds are calibrated separately by embodiment and data source.

Cross-embodiment action canonicalization. To make action channels physically consistent across embodiments, we define common right-handed body and gripper frames. The body axes point forward, left, and up. At the open-gripper zero pose, $+z$ points from the mounting flange to the gripper tip, $+y$ from the right finger to the left, and $+x$ completes the frame. EEF poses are expressed in a canonical shoulder-centered frame, denoted SC. The frame is shared by both arms of a bimanual robot; for a single-arm embodiment without an explicit shoulder center, the arm-root frame is used instead. For arm i , the action contains the absolute EEF position ${}^{SC}p_{\text{EEF},t}^{(i)} \in \mathbb{R}^3$, a continuous 6D orientation representation $r_t^{(i)} \in \mathbb{R}^6$ [58], and normalized gripper closure $g_t^{(i)} \in [0, 1]$, where 0 and 1 denote fully open and fully closed, respectively:

$$a_t^{(i)} = \left[\left({}^{SC}p_{\text{EEF},t}^{(i)} \right), \left(r_t^{(i)} \right), g_t^{(i)} \right]^\top \in \mathbb{R}^{10}, \quad a_t = \left[\left(a_t^{(L)} \right), \left(a_t^{(R)} \right) \right]^\top \in \mathbb{R}^{20}. \quad (2)$$

Single-arm samples use the 10-dimensional form. Let B be the source body frame, ${}^{SC}T_B$ the embodiment-specific rigid transform from B to SC, and ${}^B T_{\text{EEF},t}^{(i)}$ the source EEF target pose. Canonicalization is

$${}^{SC}T_{\text{EEF},t}^{(i)} = {}^{SC}T_B {}^B T_{\text{EEF},t}^{(i)}, \quad r_t^{(i)} = \rho_{6D} \left({}^{SC}R_{\text{EEF},t}^{(i)} \right). \quad (3)$$

Table 2: Open-source datasets retained after curation.

Data source	Hours	Share
RoboDesk (Open real-world): 3,235 h (21.5%)		
AgiBotWorld-Beta [1]	2,174	14.4%
Galaxea Open-World [15]	333	2.2%
RoboCOIN [45]	311	2.1%
LET [20]	172	1.1%
DROID [18]	164	1.1%
RoboMIND [44]	81	0.5%
Open simulation: 4,295 h (28.5%)		
InternData-A1 [41]	3,967	26.3%
Genie Sim 3.0 [52]	213	1.4%
RoboTwin 2.0 [31]	115	0.8%
Open-source total: 7,530 h (50%)		

The rotation and translation blocks of ${}^{SC}T_{EEF,t}^{(i)}$ provide the canonical orientation and metric absolute position target, respectively; the translation is not a temporal displacement.

Hierarchical task annotation. Each episode receives an overall goal in `global_task`, while `curr_subtask` labels the interval in which each execution stage is active. The auxiliary `scene` field describes task-relevant objects and their spatial arrangement, and `skill` names the manipulation primitive. The resulting corpus pairs synchronized observations and canonical EEF actions with episode-level goals, time-aligned subtask labels, and scene and skill context, supporting mixed-granularity instruction conditioning during training.

3 Meituan-Robotics-0

Meituan-Robotics-0 aims to transfer broad priors learned from heterogeneous multimodal data to precise and robust real-robot control. This transition requires the model to preserve its visual-semantic representations while adapting its action interface and supervision to continuous cross-embodiment control and policy-induced error states. We first describe the policy architecture in Section 3.1 and then present the three-stage training pipeline in Section 3.2: discrete multimodal pre-training, continuous cross-embodiment mid-training, and closed-loop real-world policy optimization.

3.1 Policy Architecture

MR0 formulates low-level desktop manipulation as reactive action-chunk prediction from the current observation. At this level, the policy maintains no temporal history and conditions only on the inputs available at the current decision step. Given multi-view observations $o_t = \{I_t^{(c)}\}_{c=1}^C$, language instruction l , and proprioceptive state s_t at time t , MR0 predicts an H -step action chunk

$$\pi_{\theta}(a_{t:t+H-1} \mid o_t, l, s_t). \quad (4)$$

Following the common VLA design [4, 32, 35], MR0 realizes this mapping with a shared vision–language backbone and two stage-specific output heads, as summarized in Figure 1. The shared backbone is initialized from Qwen3.5-4B [36] and remains trainable throughout the pipeline. During pre-training, FAST [33] applies a discrete cosine transform along the temporal axis and compresses the resulting coefficients with byte-pair encoding. The encoded action chunks share the backbone vocabulary and are predicted by the same autoregressive token head used for text.

For mid-training and all downstream stages, MR0 replaces the discrete interface with a continuous MMDiT action head [11, 4]. As illustrated in Figure 5, the head jointly processes an observation stream containing projected backbone features and a proprioceptive-state token, and an action stream containing a noisy action chunk. Joint self-attention exchanges information between the two streams, while the flow timestep modulates block normalization through AdaLN. The state token remains on the observation side, providing condition-side proprioceptive context throughout action generation.

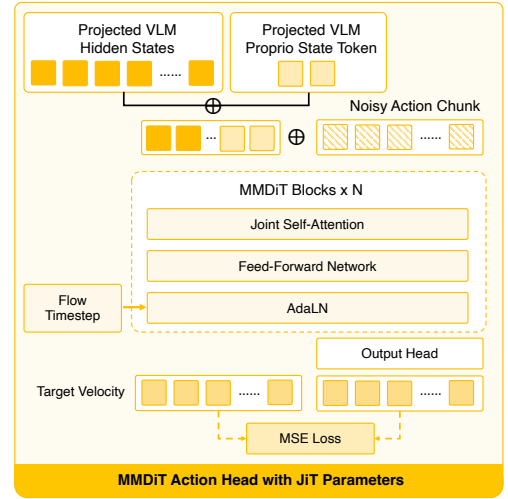


Figure 5: **Continuous action head.** MMDiT denoises action chunks through flow matching.

3.2 Progressive Training

MR0 is trained progressively to separate three sources of difficulty: heterogeneous multimodal learning, continuous cross-embodiment control, and closed-loop improvement under policy-induced errors.

3.2.1 Discrete Multimodal Pre-Training

The shared token interface allows robot trajectories and vision–language samples to use the same autoregressive objective without a separate action decoder. FAST action tokens occupy a dedicated vocabulary block whose embeddings are initialized with the mean of the original embedding table. The remaining backbone parameters are inherited and trained end-to-end, and all camera views share the native vision encoder and aspect-preserving smart-resize policy.

Each optimization step draws one batch from the VLA action pool and one from the vision–language pool. The joint objective is

$$\mathcal{L}_{\text{pre}} = \lambda_{\text{vla}} \mathcal{L}_{\text{action}} + \lambda_{\text{vlm}} \mathcal{L}_{\text{vqa}}, \quad \lambda_{\text{vla}} = 1.0, \quad \lambda_{\text{vlm}} = 0.1, \quad (5)$$

Table 3: **Pre-training data mixture.** Within-pool ratios are computed from frame counts for the VLA action pool and sample counts for the VLM co-training pool. The reported scales correspond to the effective action pool after stage-specific filtering and therefore differ from the full 15,060-hour corpus summarized in Section 2.2.

Training pool	Data category	Scale	Within-pool ratio
VLA action pool	RoboDesk (open real)	310.7M frames / 3.0k h	21.5%
	Licensed real	746.3M frames / 6.9k h	51.7%
	Synthetic	387.5M frames / 3.6k h	26.8%
	Total	1,444.5M frames / ~13.6k h	100.0%
VLM co-training pool	Real-robot grounding	66.7M samples	44.2%
	General VQA / captioning	50.6M samples	33.6%
	Spatial reasoning	11.9M samples	7.9%
	Real-robot affordance	8.2M samples	5.5%
	Simulation VQA	8.2M samples	5.4%
	Task planning / embodied QA	1.8M samples	1.2%
	Real-robot captioning	1.7M samples	1.1%
	Xingzhi VQA	1.6M samples	1.1%
	Total	150.7M samples from 64 sub-datasets	100.0%

where both terms are next-token cross-entropy over their respective targets. The action loss grounds the backbone in robot behavior, while the VQA loss preserves perceptual and semantic supervision. Table 3 reports the stage-specific mixture. Its 13.6k-hour VLA pool is distinct from the complete 15,060-hour curated inventory in Section 2.2. We train the backbone and extended action-token embeddings end to end for 500,000 steps on 256 GPUs using AdamW with cosine learning-rate decay. The resulting checkpoint initializes mid-training. Because FAST tokenization [33] introduces action-reconstruction error, downstream stages discard the token interface in favor of continuous prediction.

3.2.2 Continuous Cross-Embodiment Mid-Training

The second stage retains the token-pre-trained backbone and initializes the MMDiT action head [11] from scratch. This transfers the visual-semantic and behavioral representation learned during pre-training while replacing quantized action generation with continuous control.

Continuous action objective. We train the MMDiT head with the JiT clean-sample parameterization [26]. Following flow matching [27], a clean chunk a_0 is perturbed into a noisy sample a_τ , from which the head predicts $\hat{a}_0$ rather than velocity directly. Prediction and target are converted to velocity form using a shared clamped denominator d_τ with $t_\epsilon = 0.05$, and the model is optimized by mean squared error.

Unified action representation. To apply the continuous objective across heterogeneous embodiments, MR0 maps each native action to the canonical end-effector convention defined in Section 2.2. Each action chunk has a fixed width $d_{\max} = 20$, corresponding to two 10-dimensional end-effector actions, while a binary mask $m \in \{0, 1\}^{d_{\max}}$ identifies the valid channels for each embodiment:

$$\mathcal{L}_{\text{masked}} = \sum_{h=1}^H \sum_{j=1}^{d_{\max}} m_j \ell_{h,j} \Big/ H \sum_{j=1}^{d_{\max}} m_j, \quad (6)$$

where $\ell_{h,j}$ is the element-wise JiT loss. Padded channels produce no gradient. For a downstream embodiment with a different action width, only the dimension-dependent projections are reinitialized.

In this stage, we train on 128 GPUs using AdamW with cosine learning-rate decay, applying a lower learning rate to the transferred backbone and a higher rate to the newly initialized action head. The training mixture starts from the data pools summarized in Table 3. We rebalance it by removing a 340-million-frame synthetic sub-corpus, retaining 10% of each of the two largest general-VQA sources, and sampling VLA datasets in proportion to the square root of their frame counts. To expose the model to different levels of linguistic abstraction, we sample the local subtask label or the episode-level goal with equal probability when both are available and otherwise use the available annotation. This langmix strategy aligns the same action targets with both fine-grained commands and task-level prompts encountered downstream.

3.2.3 Closed-Loop Real-World Policy Optimization

To improve real-world robustness and reduce compounding errors during closed-loop execution, we further optimize the mid-training checkpoint using real-robot interaction data. We first fine-tune MR0 on successful demonstrations

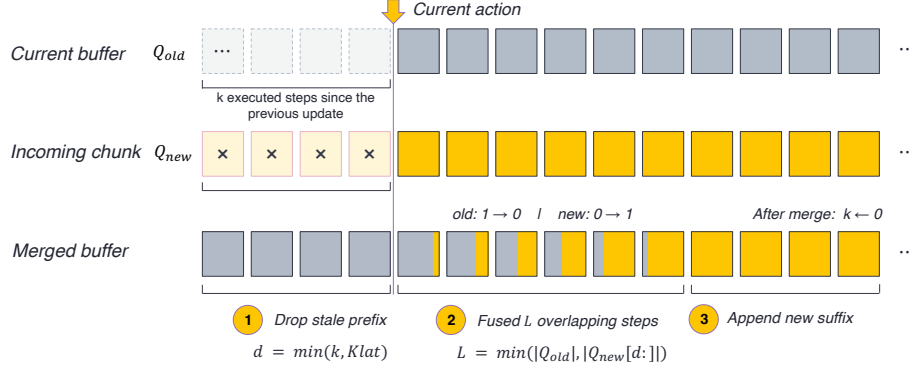


Figure 6: **Asynchronous action-chunk merging.** Predictions made stale during inference are removed from the chunk; the aligned overlap is blended with the residual buffer, and the suffix is appended for continuous execution.

and then study two closed-loop post-training methods: human-gated DAgger [17], which collects corrective actions in policy-induced states, and RECAP [37], which trains on the same demonstration and intervention data and additionally learns from outcome-labeled autonomous rollouts.

Human-gated DAgger. We first fine-tune π_{mid} on successful teleoperated demonstrations to obtain the initial task policy π_0 . We then use the human-gated component of HG-Dagger [17] to collect corrective data. The policy controls the robot by default, and an operator takes over when execution stalls or becomes incorrect or unsafe. Only human-controlled frames are retained and added to the demonstrations from all preceding rounds. Each round reinitializes the policy from π_{mid} and trains on the cumulative dataset, so information is carried by the data rather than the preceding policy weights.

RECAP-based policy optimization. In RECAP, complete autonomous episodes, including failures, are added to the demonstration and human-intervention data used by HG-Dagger. These rollouts require no operator in the loop and can therefore be collected during routine evaluation runs. Each nonterminal step receives a reward of -1 , successful termination receives 0 , and failed termination incurs a penalty of $-T_{\text{max}}$. The normalized return supervises a value model $V(o_t, l) \in [-1, 0]$ trained with value-bin cross-entropy, where larger values indicate greater proximity to successful completion. For each valid action-chunk start, an N -step advantage combines the accumulated transition reward with the predicted value change over the same horizon. When this advantage exceeds the threshold, the corresponding training prompt receives the text condition Advantage: positive; demonstration and human-intervention frames are also treated as positive. Advantage changes only the prompt routing, while the RECAP policy retains the same masked flow-matching action loss. Each policy iteration is reinitialized from π_{mid} .

At inference, base and positive-conditioned predictions are denoised from shared initial noise and combined through action-space classifier-free guidance. The resulting continuous end-effector chunk is executed by the asynchronous controller described in Section 4.1. Closed-loop data aggregation, value and policy learning, advantage labeling, condition dropout, guidance, and training hyperparameters are detailed in Section C.

4 Embodied Manipulation System

MRO augments the trained VLA policy with an inference-time system for continuous control and long-horizon task execution. An asynchronous execution layer aligns variable-latency action predictions with robot control, as detailed in Section 4.1. At a higher level, an event-driven agent retains task-relevant state beyond the policy’s current observation. As summarized in Figure 7, motor control remains with the VLA policy, and high-level planning is activated only by events that alter task progress or feasibility.

4.1 Asynchronous Policy Execution

Action chunks amortize policy inference over multiple control steps, but inference latency requires coordinating chunk generation with ongoing execution [57, 5, 6]. Our EVA-Client-based stack [47] keeps control continuous by decoupling the two: a background policy thread predicts a chunk from the latest observation at target rate f_π , while the control loop consumes one buffered command at rate f_c . If inference and sequential inverse kinematics exceed the nominal policy period, updates arrive at a lower rate.

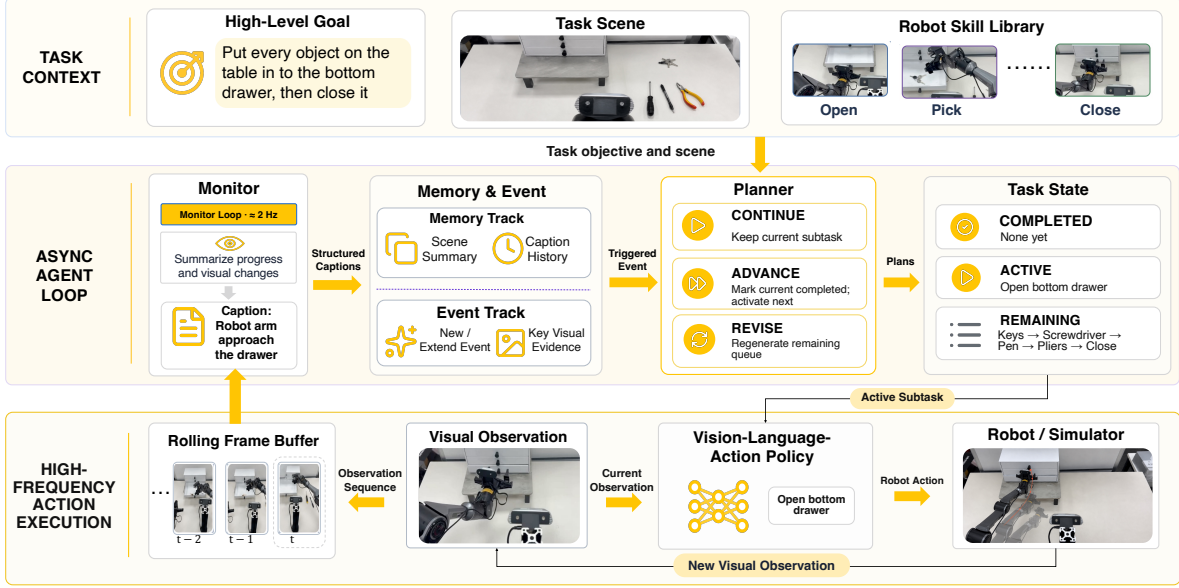


Figure 7: **Event-driven embodied-agent framework.** The asynchronous agent loop converts visual observations into structured captions, consolidates them into events, and triggers the Planner when task-state reassessment is required; the active subtask conditions the high-frequency VLA control loop.

This concurrency creates a temporal offset. By the time a new chunk Q_{new} arrives, the controller has consumed k commands since the previous successful update. We denote the unexecuted remainder of the current buffer by Q_{old} . Following the *temporal chunk-wise smoothing* of χ_0 [53], we align the two sequences as shown in Figure 6. We first drop $d = \min(k, K_{\text{lat}})$ leading commands and compute the remaining overlap:

$$\bar{Q}_{\text{new}} = Q_{\text{new}}[d:], \quad L = \min(|Q_{\text{old}}|, |\bar{Q}_{\text{new}}|). \quad (7)$$

Here K_{lat} is a step-domain cap, set as $K_{\text{lat}} \approx \lceil f_c \hat{\tau}_{\text{e2e}} \rceil$, where $\hat{\tau}_{\text{e2e}}$ is the delay from observation capture to insertion of the processed chunk into the buffer, including communication, policy inference, post-processing, and inverse kinematics.

For each position $i = 0, \dots, L - 1$ in the overlapping region, the updated command is

$$q_i^{\text{fused}} = (1 - \alpha_i)q_i^{\text{old}} + \alpha_i \bar{q}_i^{\text{new}}, \quad \alpha_i = \frac{i}{\max(L - 1, 1)}. \quad (8)$$

The unused new suffix fills the future buffer, and k is reset after a successful update. The complete procedure and concrete deployment configuration are given in Section B.

The coupled runtime parameters are f_c , f_π , and K_{lat} . When the inference pipeline keeps pace, roughly f_c/f_π commands are consumed between updates. Increasing f_π can refresh predictions more frequently, but an overly high update rate can repeatedly restart the transition from the current buffer and make the physical response sluggish. Conversely, a low update rate can exhaust the buffer, while K_{lat} trades stale-prefix removal against the remaining executable horizon. We use $f_c = 30$ Hz and target $f_\pi = 3$ Hz, and K_{lat} is tuned in control steps for the deployed serving stack.

4.2 Event-Driven Embodied Agent

The chunk merger stabilizes local policy execution but does not preserve information across subtasks or determine when a high-level plan should change. The embodied agent addresses these requirements through explicit task state, persistent event memory, and event-triggered planning.

Task decomposition and execution. Given a user goal g , initial scene context s_0 , and skill library $\mathcal{S}$, the Planner first produces an ordered sequence of executable subtasks $Q^{(0)} = \mathcal{P}_{\text{plan}}(g, s_0, \mathcal{S})$. During execution, the agent maintains the task state $X_t = (q_t, R_t)$, where q_t is the active subtask and R_t is the remaining sequence. The active subtask serves as the language instruction to the low-level policy, which generates actions as $\pi_{\text{VLA}}(o_t, q_t)$. It remains active until the

agent observes sufficient evidence of completion, failure, or a change in task feasibility. This state representation lets the VLA policy execute continuously while the Planner operates only at task-level transition points.

Visual monitoring. The Monitor tracks local execution by mapping a short observation window W_t and the active subtask to a structured caption $C_t = \mathcal{M}(W_t, q_t)$. Conditioning on q_t focuses the description on relevant objects, spatial relations, interactions, and execution progress. The temporal window captures changes such as grasping, placement, or object removal that may be ambiguous in a single frame. The Monitor does not decide whether a subtask has finished or modify X_t ; it only produces visually grounded evidence for event construction.

Event construction and memory. Monitor outputs enter a caption buffer in observation-time order. At each memory update, B_t contains the captions that have not yet been processed, and the persistent history $\mathcal{H}_{<t}$ stores ordered event summaries together with their supporting keyframes. The Memory Manager converts the new captions into an event and determines whether that event warrants high-level reassessment:

$$(E_t, z_t) = \mathcal{G}(B_t, \mathcal{H}_{<t}, X_t, g), \quad z_t \in \{0, 1\}. \quad (9)$$

When the new evidence continues the same interaction, it is merged into the latest memory entry; a distinct state change creates a new entry. For each event, the keyframe set K_t is obtained by uniformly sampling frames from the observation windows associated with its captions. Memory insertion is independent of planner activation: routine observations can be recorded with $z_t = 0$, while subtask completion, execution failure, and changes in task preconditions set $z_t = 1$. This separation preserves information that may become relevant to a later instruction without repeatedly invoking the Planner over an unchanged state.

Event-triggered planning. The Planner performs two related operations. $\mathcal{P}_{\text{plan}}$ constructs or revises a subtask sequence from the goal, scene context, task state, and available skills. When $z_t = 1$, $\mathcal{P}_{\text{decide}}$ instead evaluates the current event, its keyframes, relevant entries retrieved from $\mathcal{H}_{<t}$, the task state X_t , and the user goal g . The resulting decision may preserve the active subtask, advance to the next one, wait for a temporarily unsatisfied precondition, or invoke $\mathcal{P}_{\text{plan}}$ to regenerate the remaining sequence. The Planner judges progress from the observed task state rather than requiring the complete action trajectory. When the event or its visual evidence is inconclusive, it leaves X_t unchanged.

Asynchronous coordination. The Monitor, Memory Manager, Planner, and VLA policy operate at different rates. Multiple Monitor requests may run concurrently, so their outputs can arrive out of order; captions are therefore committed according to observation time before event construction. Planner inference does not pause monitoring or low-level execution, and captions produced during planning remain buffered for the next memory update. Task-state mutations are serialized after each Planner decision, preventing concurrent decisions from modifying the same execution state. The system consequently preserves continuous perception and control while keeping event memory and task transitions temporally consistent.

5 Experiments

5.1 Simulation

RoboTwin2.0. RoboTwin2.0 [31] is a bimanual desktop-manipulation benchmark comprising 50 tasks with a 14-dimensional joint-space action interface. Each task provides 50 demonstrations in the clean-scene split (`demo_clean`) and 500 demonstrations in the randomized split (`demo_randomized`), which varies the background, table clutter, table height, and lighting. We jointly fine-tune all 50 tasks end-to-end from the stage-2 checkpoint using native-resolution inputs. Evaluation covers 100 episodes per task on each split, resulting in 5,000 episodes per split. We evaluate the VLA policy alone, without the agent system or RL post-training.

As shown in Table 4, Meituan-Robotics-0 achieves 93.6% overall success, placing it at the top of the comparison alongside InternVLA-A1.5, while its Clean and Randomized results differ by only 0.5 percentage points. The comparison with ABot-M0 is particularly informative: both methods predict clean actions and optimize a velocity-space loss, yet MR0 achieves 7.8 points higher overall success. Since MR0 adapts a shared stage-2 checkpoint learned from heterogeneous data and embodiments, these results demonstrate *effective transfer*

Table 4: **RoboTwin 2.0 results.** Success rate (%) over 50 tasks and 100 episodes per task in each split. Overall is the mean of Clean and Randomized; “–” denotes an unreported result. Bold and underlining indicate the best and second-best results.

Method	Clean	Random.	Overall
π_0 [4]	65.9	58.4	62.2
$\pi_{0.5}$ [3]	82.7	76.8	79.8
GR00T N1.6 [32]	47.6	–	–
X-VLA [46]	72.8	72.8	72.8
ABot-M0 [49]	86.1	85.1	85.6
Qwen-VLA-Instruct [35]	86.1	87.2	86.7
LingBot-VLA [43]	86.5	85.3	85.9
Motus [2]	88.7	87.0	87.8
StarVLA [39]	88.7	87.8	88.3
InternVLA-A1 [8]	89.4	89.6	89.5
Being-H0.7 [29]	90.2	89.6	89.9
Fast-WAM [54]	91.9	91.8	91.8
LingBot-VA [24]	92.9	91.5	92.2
InternVLA-A1.5 [34]	93.3	93.0	93.2
Meituan-Robotics-0	<u>93.2</u>	93.9	93.6

Table 5: **RoboDojo simulation results.** Results are reported as Score/SR (%). Models marked with * use additional RoboDojo-oriented enhancements: DM0.5 uses a memory-enabled RoboDojo checkpoint; G0 and G0.5 use chain-of-thought reasoning, with G0.5 additionally using visual-history memory; Hy-Embodied-0.5-VLA combines visual-history memory with UMI pre-training; and Xiaomi-Robotics-1 uses UMI pre-training. MR0 uses no additional benchmark-specific enhancement. Bold indicates the best Score or SR in each column.

Model	Avg. $\uparrow$	Gen-Std $\uparrow$	Gen-Rand $\uparrow$	Precision $\uparrow$	Long-Horizon $\uparrow$	Memory $\uparrow$	Open $\uparrow$
WAM							
AHA-WAM [7]	4.82/2.39	10.32/6.00	1.26/0.00	5.86/2.42	8.61/2.67	2.97/2.78	0.88/0.83
GigaWorld-Policy-0 [50]	6.20/3.27	10.28/6.00	0.41/0.00	6.15/1.83	15.51/8.92	3.46/2.22	0.54/0.50
X-WAM [12]	7.69/3.83	11.24/5.00	3.54/1.00	6.72/1.83	17.47/9.08	6.32/4.67	0.57/0.25
VLA							
GalaxeaVLA (G0)* [16]	5.82/2.96	8.71/6.00	0.36/0.00	8.10/3.83	12.60/5.58	3.17/1.89	0.70/0.67
VLAct [38]	10.66/7.60	16.33/12.00	2.74/1.00	20.62/15.25	20.12/13.67	0.66/0.56	2.37/2.25
InternVLA-A1.5 [30]	11.15/7.14	16.81/12.00	3.90/2.00	15.23/10.17	23.80/13.75	4.93/3.56	1.43/1.42
$\pi_{0.5}$ [14]	11.41/6.91	20.93/15.00	5.82/1.00	12.40/5.50	23.54/14.67	5.78/4.56	1.98/1.67
Spatial Forcing [21]	12.38/8.04	21.25/15.00	6.98/4.00	17.33/10.58	23.26/14.58	5.43/4.11	1.78/1.58
Hy-Embodied-0.5-VLA* [55]	13.07/8.80	21.98/17.00	1.57/0.00	13.81/8.00	25.74/14.92	13.37/12.11	0.65/0.58
Meituan-Robotics-0	14.95/9.53	19.28/12.67	8.22/3.67	16.77/7.75	29.61/18.58	10.06/8.89	4.54/4.25
Xiaomi-Robotics-1* [40]	20.07/13.93	35.65/28.00	11.44/6.00	26.69/18.83	38.39/23.67	7.81/6.56	3.94/3.58
GalaxeaVLA (G0.5)* [28]	20.23/14.88	26.74/20.00	11.16/6.00	28.25/20.42	44.12/32.25	8.61/7.33	1.73/1.58
DM0.5* [9]	24.90/19.34	23.49/18.00	8.06/4.00	24.82/16.75	33.70/19.50	47.74/47.44	2.43/2.08

to precise bimanual control and show that action-head parameterization alone does not determine downstream performance.

RoboDojo. RoboDojo [9] evaluates generalist manipulation policies under a unified protocol spanning simulation and real-world settings. Its simulation track comprises 42 bimanual tasks across five capability dimensions: Generalization, Precision, Long-Horizon, Memory, and Open-vocabulary instruction following. Following the official protocol, we evaluate the standalone Meituan-Robotics-0 VLA policy without the embodied-agent system and RL-based post-training. Table 5 reports the resulting Score and Success Rate.

MR0 achieves an average Score of 14.95 and an SR of 9.53%, ranking first among VLA models that use no benchmark-specific enhancement and fourth across all evaluated policies. It outperforms every WAM policy and notably surpasses Hy-Embodied-0.5-VLA, which incorporates UMI pre-training and visual-history memory. The model also ranks third on Memory and fourth on Long-Horizon. These results suggest that *broad visual-task priors and reliable local control provide substantial temporal robustness when task state remains visually recoverable, although genuinely ambiguous states still require explicit memory.*

Its strongest relative result is on Open-vocabulary instruction following, where it ranks first with a Score/SR of 4.54/4.25. This advantage indicates stronger transfer to underrepresented tasks than competing methods, while the low absolute scores across all models show that open-vocabulary generalization remains challenging. Broader pre-training, including UMI data, may improve task coverage but does not resolve failures on poorly represented behaviors. On the Align_blocks task, for example, models frequently default to grasping the box rather than performing the requested alignment. This behavior reveals a *persistent bias toward familiar action patterns over instruction-specific intent*, leaving task-level generalization as a central challenge for future VLA research.

5.2 Real-Robot Evaluation

We evaluate Meituan-Robotics-0 at two complementary levels. The policy evaluation quantifies manipulation performance and isolates the effects of human-intervention data and RECAP. The agent evaluation examines two history-dependent behaviors that are not captured by task-level success rates: retrieving a state that is no longer observable and resuming an interrupted objective when its execution condition is restored.

5.2.1 Experimental Setup

Tasks and metrics. To evaluate semantic grounding, deformable-object manipulation, and precision bimanual control in real-world settings, we design three complementary tasks (Figure 8):

- **Letter-block placement.** The policy must identify the block carrying the next letter, grasp and reorient it, and place it in the target groove to spell “MEITUAN” in order. The task evaluates semantic grounding, orientation control, and long-horizon sequential execution. A trial succeeds only when all seven blocks are placed in the correct order and orientation. If $\bar{k}$ is the mean number of consecutively correct placements before the first error, then $PS_{\text{block}} = \bar{k}/7$.
- **Garment folding.** The policy must reposition and spread the garment, fold it into the target shape, and straighten the final configuration. The task evaluates coordinated bimanual manipulation and continuous shape control over a deformable object. Success requires the target folded configuration, and execution efficiency is measured as $PS_{\text{fold}} = \text{clip}((300 - T)/210, 0, 1)$, where T is the average completion time.
- **ethernet insertion.** The policy must grasp the cable, transfer it between the two arms, align the connector with the port, and complete insertion. The task evaluates localization of a thin deformable object, bimanual handover, and precise position and orientation control. Progress is divided into cable grasping, bimanual handover, and connector insertion. If $\bar{m}$ is the mean number of completed milestones, then $PS_{\text{ethernet}} = \bar{m}/3$; success requires full insertion.

Each task is paired with a binary success criterion and a task-specific progress score (PS) that captures partial completion or execution efficiency. Across all tasks, each policy is evaluated for 30 trials without human intervention. We report task-level success rate (SR) and PS, together with their macro-averages. Policies within each comparison use matched initial conditions and the same rollout protocol.



Figure 8: **Real-robot evaluation tasks.** Representative stages of (a) letter-block placement, (b) garment folding, and (c) Ethernet insertion.

Compared policies and post-training data. We compare MR0 with $\pi_{0.5}$ [3]. For each backbone, we evaluate an SFT policy trained on successful teleoperation demonstrations and an HG-Dagger policy trained on the same demonstrations together with accumulated human-intervention trajectories. The final HG-Dagger checkpoints are used for the matched backbone comparison. Both models use the same task data, observation and action representations, preprocessing pipeline, and inference settings within each training stage.

All policies predict 50-step chunks in a 20-dimensional bimanual end-effector action space. For the HIL scaling study, each round is independently initialized from the same mid-training checkpoint and trained on the complete demonstration set together with all interventions collected up to that round. Round 0 is therefore an independently trained demonstration-only reference rather than the SFT checkpoint used in the main comparison. Further data-preparation details are provided in Section A.

Table 6 summarizes the post-training data. Demonstrations and intervention episodes are used by both HG-Dagger and RECAP. Outcome-labeled autonomous rollouts are collected for Ethernet insertion and used only by RECAP. Intermediate checkpoints provide the policies evaluated in the HIL scaling study (Figure 11).

5.2.2 Standalone Policy Evaluation

Table 6: **Real-robot post-training data statistics.** Demo and HIL denote teleoperated demonstrations and episodes containing human interventions. Outcome-labeled autonomous rollouts are used only by RECAP. Valid Starts counts training samples retained after static-interval trimming and control-boundary filtering.

Task	Demo	HIL	Rollout (Succ)	Rollout (Fail)	Total	Valid Starts
Letter-block placement	500	200	—	—	700	~900K
Garment folding	400	400	—	—	800	~1.1M
Ethernet insertion	200	802	279	389	1670	~1.5M

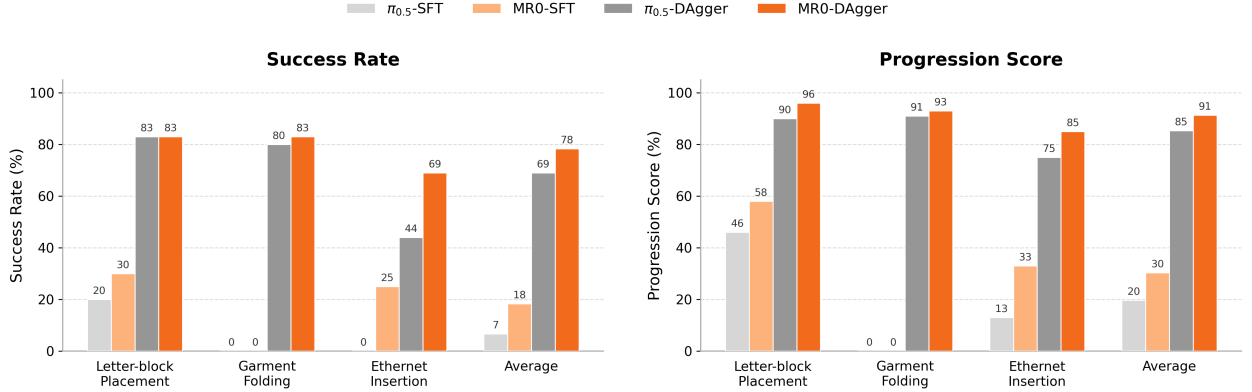


Figure 9: Real-robot SR and PS of $\pi_{0.5}$ and MR0 after SFT and HG-Dagger post-training. Average denotes the macro-average over the three tasks.

Main comparison. Under matched HG-Dagger post-training, MR0 achieves a macro-average SR of 0.78 and PS of 0.91, compared with 0.69 and 0.85 for $\pi_{0.5}$ (Figure 9). The largest difference occurs on Ethernet insertion, where MR0 improves SR by 0.25 and PS by 0.10. On letter-block placement, both models attain the same SR, while MR0 completes more placements before failure. On garment folding, the SR difference is small, but MR0 reduces average completion time from 133 to 117 seconds. These results indicate that *MR0’s main advantage lies in precision-sensitive alignment and insertion, while its gains on the other tasks appear primarily in completion quality and efficiency.*

Effect and scaling of human-intervention data. Adding intervention trajectories substantially improves both backbones. For $\pi_{0.5}$, macro-average SR/PS increases from 0.07/0.20 to 0.69/0.85; for MR0, it increases from 0.18/0.30 to 0.78/0.91 (Figure 9). Both SFT policies fail all garment-folding trials and make limited progress on Ethernet insertion, indicating that *successful demonstrations alone provide insufficient coverage of the failure and recovery states encountered during closed-loop execution.*

The scaling results in Figure 11 show substantial early gains from additional HIL data. By Round 4, SR rises from 0.23 to 0.83 on letter-block placement and from 0.00 to 0.83 on garment folding. Both tasks then plateau, with unchanged SR and slightly lower PS at Round 5. Ethernet insertion improves throughout the available rounds and reaches 0.69 SR and 0.85 PS. Overall, *HIL data deliver substantial early gains, but their marginal benefit diminishes once task performance begins to saturate.*

RECAP-based policy optimization. (RollBack). We evaluate RECAP on Ethernet insertion, the task with the strongest precision requirements and the largest gap between partial progress and successful completion. HG-Dagger already learns cable grasping and bimanual handover from demonstrations and interventions, but its remaining failures are concentrated at connector alignment and final insertion. Following Section 3.2.3, both methods are independently initialized from π_{mid} . At each data scale, they use the same demonstrations and interventions, while RECAP additionally incorporates outcome-labeled autonomous rollouts for value learning and advantage conditioning. This protocol keeps the human-data budget matched while evaluating RECAP as an end-to-end post-training procedure rather than a sample-matched objective ablation.

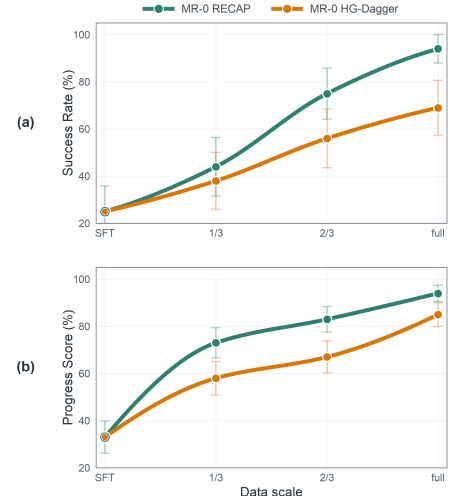


Figure 10: RECAP versus HG-Dagger on

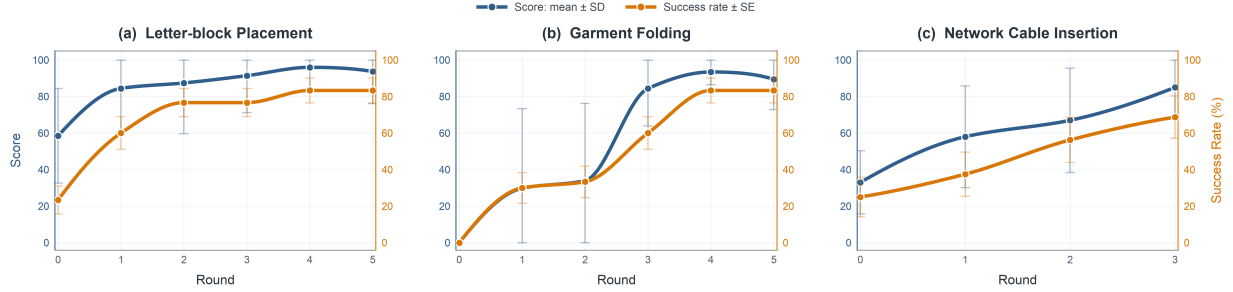


Figure 11: HIL data scaling on the three real-robot tasks. Round 0 uses teleoperation demonstrations only; each subsequent round adds all intervention data accumulated up to that round. Every policy is independently initialized from the same mid-training checkpoint.

RECAP improves both metrics at every nonzero scale (Figure 10). At full scale, it reaches 0.94 SR and 0.94 PS, compared with 0.69 and 0.85 for HG-Dagger, corresponding to gains of 0.25 SR and 0.09 PS. The larger improvement in SR shows that both methods often reach the later stages of the task, but RECAP more consistently converts alignment attempts into completed insertions. Its SR advantage also widens as more data are incorporated, suggesting that *outcome feedback becomes increasingly useful once the policy encounters a broader range of self-induced failure states*. Using HG-Dagger for letter-block placement and garment folding and RECAP for Ethernet insertion, the final task-specific configuration reaches a macro-average SR of 0.86 and PS of 0.93.

5.2.3 Embodied Agent Evaluation

The policy experiments measure execution within a single task, but do not test whether the system can recover information that has disappeared from the current observation or preserve an objective across an interruption. We therefore conduct two controlled qualitative deployments: cross-task event retrieval and condition-aware task resumption.

Agent setup. The Monitor, Memory Manager, and Planner use Qwen3.5-35B-A3B and share an SGLang inference pool with four instances. The Monitor runs at 2 Hz over an eight-frame, two-second observation window. Each persistent event stores a semantic summary and four to eight archived keyframes.

Cross-task event retrieval. In Figure 12, the initial instruction is to place the tabletop objects in the bottom drawer and close it. After the key has been placed and the screwdriver subtask has begun, a person moves the key to the middle drawer. The Monitor captures this state change, and the Memory Manager stores the event with its keyframes. Because the active screwdriver subtask remains feasible, execution continues and the initial instruction is completed. When the user later requests the key, neither the closed cabinet nor the empty table reveals its location. The agent retrieves the stored event, identifies the middle drawer as the key’s last observed location, and generates a plan to recover it. This case shows that event summaries and archived visual evidence can support a new task when the relevant state is no longer directly observable.

Interruption and condition recovery. In Figure 13, the agent is instructed to place the key in the bottom drawer and close it. A person removes the key after the drawer is opened but before the grasp is completed. The Monitor detects that the target is unavailable, while the Memory Manager records the condition change. The Planner retains the pending placement subtask and enters a waiting state rather than discarding or replanning the objective. When the key returns, a recovery event triggers reassessment against the latest observation, after which the robot places the key and completes the original instruction. This case demonstrates that persistent task state can preserve intent during temporary infeasibility and resume execution when the missing condition is restored.

Together, the two cases exercise complementary forms of temporal context. *Event memory and persistent task state extend planning beyond the current observation by supporting cross-task retrieval and interrupted-task resumption.*

6 Conclusion

7 Contributions

All people are cataloged **alphabetically by first name**. (†) indicates the project leader and (‡) indicates the sponsors.

Contributors:

Caicheng Wang	Jiacong Zhou	Shuo Li	Yang Yi	Zilong Huang
Chen Zhao	Jie Hu [†]	Tianyi Zhang	Yanli Tan	Zipeng Ye
Chengpeng Jin	Jiyuan Ru	Xi Zhang	Yifu Xu	Ziqin Wang
Duo Xu	Linjiang Huang	Xin Chen	Yimeng Jia	Ziyuan Qin
Fengjiao Chen	Manyuan Zhang	Xuezhi Cao	Yuchen Tang	
Guanglu Wan [‡]	Ningchu Liu	Xunliang Cai [‡]	Yuchen Xie [‡]	
Heqing Yang	Shijie Zhou	Yanbin Zeng	Yuxiang Zheng	
Jia Wang	Si Liu	Yang Yang	Zerwei Chng	

A Real-Robot Setup and Data Preparation

Data interface across teleoperation, HIL, and autonomous rollouts. Our bimanual AgileX PiPER setup contains two human-operated leader arms and two corresponding task-executing follower arms. The operator moves the leader arms, and their joint positions are relayed as target commands to the follower arms. Each trajectory records three camera streams from the task workspace, the observed follower-arm configuration in `observations/qpos`, and the robot command in `action`. During teleoperation and human intervention, `action` is constructed from the leader-arm joint positions and sent to the follower arms; during autonomous execution, it is produced by the policy. Thus, leader-arm state is not a policy observation, but it does determine the supervised action target for human-controlled frames. Both control sources use the same stored camera, state, and action interface.

The demonstration dataset $\mathcal{D}_{\text{demo}}$, collected by teleoperation, contains only successful executions. The tasks and data quantities used in our experiments are specified in Section 5.2.

Real-robot observation and action representation. The policy receives synchronized images from three RGB cameras: one head camera and two wrist cameras. Each camera captures images at a native resolution of 640×480 . The policy uses a single observation frame and does not include an image history.

For these real-robot experiments, we do not warp the images to a fixed square resolution. We instead use the Smart Resize operation provided by Qwen3.5-VL, which limits the number of visual tokens while preserving the original aspect ratio. Under this task-specific configuration, images are resized to approximately 352×256 before visual encoding. We use the same image processing procedure during post-training and real-robot inference.

The policy state is derived from the observed follower-arm configuration, and the action target from the command sent to the follower arms. Both are converted to the same 20-dimensional end-effector representation. For each arm, this representation contains a 3D Cartesian position, a 6D rotation, and an absolute gripper opening value:

$$\mathbf{x}_t = [\mathbf{p}^L, \mathbf{r}_{6D}^L, g^L, \mathbf{p}^R, \mathbf{r}_{6D}^R, g^R] \in \mathbb{R}^{20}, \quad \mathbf{x}_t \in \{\mathbf{s}_t, \mathbf{a}_t\}. \quad (10)$$

The policy predicts an action chunk containing $H = 50$ control steps:

$$\mathbf{a}_{t:t+H} = [\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+49}] \in \mathbb{R}^{50 \times 20}. \quad (11)$$

Data collection, policy inference, and robot control all operate at 30 Hz. Teleoperation data, HIL data, and autonomous execution therefore use the same control frequency.

Static-interval trimming and valid starts. Real-robot trajectories contain sustained static intervals before, during, and after task execution. A transition is considered static when consecutive changes in both the observed arm configuration and commanded action remain below their respective thresholds for a minimum run length. We remove the detected intervals before constructing training samples so that hold actions do not dominate the action targets. This is a dataset-specific post-training operation, not a general component of MR0 pre-training or mid-training.

The detector is configured separately for each post-training stage and dataset because they contain different mixtures of slow human corrections, autonomous motion, and pauses. A threshold that is too small leaves idle or near-static intervals in the data, whereas one that is too large removes deliberate slow motion and useful corrective behavior. We therefore calibrate the action- and configuration-change thresholds for the relevant data mixture rather than define one numerical criterion as a general MR0 method.

After trimming and source selection, trajectories are split at episode and control-source boundaries. A frame t is a valid start only when all $H = 50$ target actions from t through $t + 49$ are real, non-tail-padded steps from the same retained segment. Consequently, an action chunk cannot cross a takeover boundary or acquire repeated hold targets through end-of-segment padding. Static-interval trimming and valid-start filtering jointly reduce the bias toward static predictions that can otherwise cause the deployed policy to halt. DAGger training uses teleoperation and human-controlled intervention segments, whereas RECAP additionally uses autonomous segments with outcome and advantage labels. All retained SFT and DAGger samples receive equal sampling weight.

Post-training implementation details. We use standard image augmentation during post-training: small random crops and rotations are applied to the head-camera view, and color jitter is applied to all three views, while wrist-camera geometry is left unchanged. State and action dimensions are quantile-normalized using their first and 99th percentile statistics; the same training-set statistics are used to unnormalize actions during deployment. These are implementation choices for the real-robot recipe rather than contributions of the method, and their effects are evaluated separately in ??.

Table 7: Concrete configuration of the asynchronous HIL executor.

Quantity or option	Value	Role
Strategy	async/linear_overlap	Background prediction with latency-aware buffer updates.
f_c (frame_rate)	30 Hz	Nominal HIL command-consumption rate; publish_rate is matched to the same value.
f_π (inference_rate)	3 Hz	Target background prediction rate.
H	50	Action horizon returned by the policy server.
K_{lat} (latency_k)	4	Maximum number of leading commands removed from an incoming chunk.
m_{min} (buffer default)	8	Minimum old-reference length used for short-buffer padding.
setup_warmup_chunks	2	Synchronous startup queries before the background loop begins.

B Asynchronous Policy Execution Details

This section gives the complete buffer-update procedure for the asynchronous controller summarized in Section 4.1. The control and inference loops share a locked action buffer and run concurrently.

Here H denotes the policy action horizon. The async preset leaves `execute_horizon` unset and uses `action_time_scale=1.0`, so the client retains all $H = 50$ returned actions before latency trimming. The two warm-up queries prime the inference and IK pipeline; the action buffer is then cleared and anchored at the current robot state, rather than prefilled with two predicted chunks.

C Closed-Loop Real-Robot Post-Training Details

C.1 HG-DAGger-Style Human Intervention

Starting from the mid-training checkpoint π_{mid} , we first train an SFT policy π_0 on the successful demonstration set $\mathcal{D}_{\text{demo}}$. During subsequent HG-DAGger-style collection, the policy controls the robot by default and a human operator takes over when the policy stalls or executes an incorrect or unsafe motion. Let $g_t \in \{0, 1\}$ denote this control gate. The executed action is

$$a_t^{\text{exec}} = (1 - g_t) \pi_{k-1}(o_t, \ell) + g_t \pi_H(o_t), \quad (12)$$

where $g_t = 0$ selects the current policy and $g_t = 1$ selects the human policy π_H . The operator retains control through a contiguous recovery segment and returns control only after reaching a state from which autonomous execution can safely continue. Only human-controlled samples are added to the HG-DAGger training set:

$$\mathcal{D}_{\text{HIL}}^{(k)} = \{(o_t, a_t^H) \mid g_t = 1\}. \quad (13)$$

After round k , the cumulative supervised dataset is

$$\mathcal{D}_{\text{DAGger}}^{(k)} = \mathcal{D}_{\text{demo}} \cup \bigcup_{i=1}^k \mathcal{D}_{\text{HIL}}^{(i)}. \quad (14)$$

We train each π_k from π_{mid} on $\mathcal{D}_{\text{DAGger}}^{(k)}$ rather than continuing from π_{k-1} . Thus, the deployed policy determines the states visited in the next collection round, while information across rounds is carried by the cumulative data rather than sequential weight updates. The policy objective remains the masked flow-matching action loss; HG-DAGger changes the collection and sample-selection procedure, not the loss itself.

C.2 RECAP Policy Optimization

RECAP is not defined as a policy update applied after the final HG-DAGger weights. For a given RECAP run, let $\mathcal{D}_{\text{base}}$ denote its supervised real-robot data, comprising demonstrations and any human-intervention data included in that run. At iteration m , we deploy the current RECAP policy autonomously, retain complete successful and failed episodes with their outcomes as $\mathcal{D}_{\text{auto}}^{(m)}$, and aggregate

Algorithm 1 Latency-Aware Asynchronous Policy Execution**Require:** Action horizon H , control rate f_c , target inference rate f_π , latency cap K_{lat} , minimum old-reference length

```

 $m_{\min}$ 
1: Shared state: buffer  $Q \leftarrow \emptyset$ , counter  $k \leftarrow 0$ , and  $q_{\text{last}} \leftarrow q_{\text{current}}$ 
2: Each pop and each complete buffer update are protected by the same lock.
3: Run the following control and inference loops concurrently.
4: Control loop (at  $f_c$ ):
5: while the system is running do
6:   if  $Q \neq \emptyset$  then
7:      $q \leftarrow \text{POPFRONT}(Q)$ ;  $k \leftarrow k + 1$ 
8:     if  $Q = \emptyset$  then
9:        $q_{\text{last}} \leftarrow q$ 
10:    end if
11:     $\text{PUBLISH}(q)$ 
12:  end if
13: end while
14: Inference loop (at target  $f_\pi$ ):
15: while the system is running do
16:    $Q_{\text{new}} \leftarrow \text{IK}(\text{PREDICT}(\text{OBSERVE}()))$   $\triangleright |Q_{\text{new}}| = H$ 
17:   if  $Q_{\text{new}} = \emptyset$  then
18:     continue
19:   end if
20:    $d \leftarrow \min(k, K_{\text{lat}})$ 
21:   if  $d \geq |Q_{\text{new}}|$  then
22:     continue  $\triangleright Q$  and  $k$  remain unchanged
23:   end if
24:    $\bar{Q}_{\text{new}} \leftarrow Q_{\text{new}}[d:]$ 
25:   if  $Q = \emptyset \wedge q_{\text{last}} \neq \emptyset$  then
26:      $Q_{\text{old}} \leftarrow [q_{\text{last}}]^{m_{\min}}$ ;  $q_{\text{last}} \leftarrow \emptyset$ 
27:   else if  $Q = \emptyset$  then
28:      $Q \leftarrow \bar{Q}_{\text{new}}$ ;  $k \leftarrow 0$ 
29:   continue
30:   else
31:      $Q_{\text{old}} \leftarrow Q$ 
32:     if  $|Q_{\text{old}}| < m_{\min}$  then
33:       Extend  $Q_{\text{old}}$  to  $m_{\min}$  by repeating its terminal command
34:     end if
35:   end if
36:    $L \leftarrow \min(|Q_{\text{old}}|, |\bar{Q}_{\text{new}}|)$ 
37:   for  $i = 0$  to  $L - 1$  do
38:      $\alpha_i \leftarrow i / \max(L - 1, 1)$ 
39:      $q_i^{\text{fused}} \leftarrow (1 - \alpha_i)q_i^{\text{old}} + \alpha_i \bar{q}_i^{\text{new}}$   $\triangleright$  Linear transition
40:     Overwrite the gripper entries of  $q_i^{\text{fused}}$  with those of  $\bar{q}_i^{\text{new}}$ 
41:   end for
42:    $Q \leftarrow [q_0^{\text{fused}}, \dots, q_{L-1}^{\text{fused}}] \parallel \bar{Q}_{\text{new}}[L:]$ 
43:    $k \leftarrow 0$ 
44: end while

```

$$\mathcal{D}_{\text{RL}}^{(m)} = \mathcal{D}_{\text{base}} \cup \bigcup_{i=1}^m \mathcal{D}_{\text{auto}}^{(i)}. \quad (15)$$

At every iteration, the value model is fit from its fixed initialization and the policy is fit from π_{mid} on the current cumulative dataset. As with HG-Dagger, the deployed policy changes across collection iterations, but the training initialization remains fixed.

Outcome rewards and value targets. Episode-level outcomes are converted into per-timestep rewards:

Algorithm 2 HG-Dagger-style real-robot post-training**Require:** Mid-training checkpoint π_{mid} , demonstration dataset $\mathcal{D}_{\text{demo}}$, rounds K

- 1: $\mathcal{D}_{\text{DAgger}}^{(0)} \leftarrow \mathcal{D}_{\text{demo}}$
- 2: Train π_0 from π_{mid} on $\mathcal{D}_{\text{DAgger}}^{(0)}$
- 3: **for** $k = 1$ to K **do**
- 4: Deploy π_{k-1} under operator gating (Equation (12)); collect $\mathcal{D}_{\text{HIL}}^{(k)}$
- 5: $\mathcal{D}_{\text{DAgger}}^{(k)} \leftarrow \mathcal{D}_{\text{DAgger}}^{(k-1)} \cup \mathcal{D}_{\text{HIL}}^{(k)}$
- 6: Train π_k from π_{mid} on $\mathcal{D}_{\text{DAgger}}^{(k)}$
- 7: **end for**

Ensure: Final task policy π_K **Algorithm 3** RECAP policy optimization**Require:** Mid-training checkpoint π_{mid} , value initialization V_{init} , base dataset $\mathcal{D}_{\text{base}}$, iterations M

- 1: $\mathcal{D}_{\text{RL}}^{(0)} \leftarrow \mathcal{D}_{\text{base}}$
- 2: Train π_0^{RL} from π_{mid} on $\mathcal{D}_{\text{RL}}^{(0)}$
- 3: **for** $m = 1$ to M **do**
- 4: Deploy π_{m-1}^{RL} autonomously; collect complete episodes and outcomes $\mathcal{D}_{\text{auto}}^{(m)}$
- 5: $\mathcal{D}_{\text{RL}}^{(m)} \leftarrow \mathcal{D}_{\text{RL}}^{(m-1)} \cup \mathcal{D}_{\text{auto}}^{(m)}$
- 6: Fit V_m from V_{init} on $\mathcal{D}_{\text{RL}}^{(m)}$ using outcome-derived targets
- 7: Label each valid start with $I_t = \mathbb{I}[A_t^{(N)} \geq \epsilon]$ under V_m
- 8: Train π_m^{RL} from π_{mid} on $\mathcal{D}_{\text{RL}}^{(m)}$, routing positive starts with c_t
- 9: **end for**

Ensure: Final task policy π_M^{RL}

$$r_t = \begin{cases} 0, & t = T \text{ and the task succeeds,} \\ -T_{\text{max}}, & t = T \text{ and the task fails,} \\ -1, & \text{otherwise,} \end{cases} \quad R_t = \sum_{t'=t}^T r_{t'}, \quad (16)$$

where T is the terminal timestep and T_{max} is the maximum episode length, which serves both as the terminal failure penalty and as the basis for normalization. Every step incurs unit cost, so the return measures the remaining time to completion, and a failed episode is charged an additional T_{max} . Normalizing by $D = 2T_{\text{max}}$ and clipping gives the scalar target

$$y_t = \text{clip} \left(\frac{R_t}{D}, -1, 0 \right) = \begin{cases} \text{clip} \left(-\frac{T-t}{D}, -1, 0 \right), & \text{success,} \\ \text{clip} \left(-\frac{T-t+T_{\text{max}}}{D}, -1, 0 \right), & \text{failure.} \end{cases} \quad (17)$$

Targets therefore lie in $[-1, 0]$: values closer to zero indicate greater proximity to successful completion, while the terminal penalty separates failed trajectories from successful ones.

Distributional value prediction. The value network predicts a categorical distribution over $B = 201$ atoms instead of directly regressing y_t :

$$p_\phi(V \mid o_t, \ell) \in \Delta^B, \quad z_i = V_{\min} + i \frac{V_{\max} - V_{\min}}{B-1}, \quad i = 0, \dots, B-1, \quad (18)$$

where $V_{\min} = -1$, $V_{\max} = 0$, and the atom spacing is $\Delta z = 0.005$. We project a scalar target onto its two neighboring atoms using

$$b_t = \text{clip}\left(\frac{y_t - V_{\min}}{\Delta z}, 0, B - 1\right), \quad l = \lfloor b_t \rfloor, \quad u = \lceil b_t \rceil. \quad (19)$$

When $l < u$, the target masses are $q_l = u - b_t$ and $q_u = b_t - l$. When $l = u$, we set $q_l = 1$; all other entries are zero. The value loss and continuous prediction are

$$\mathcal{L}_V = - \sum_{i=0}^{B-1} q_i \log p_\phi(V = z_i \mid o_t, \ell), \quad V(o_t, \ell) = \sum_{i=0}^{B-1} p_\phi(V = z_i \mid o_t, \ell) z_i. \quad (20)$$

The soft projection preserves local variation in the continuous target while retaining a classification objective.

Value-model architecture. Our value model follows the RECAP implementation in RLinf [37]. It receives the three RGB views, proprioceptive state, and task instruction. A SigLIP2-So400m/14 encoder maps each image to 256 patch tokens projected to 640 dimensions, producing 768 visual tokens across the three views. These tokens are concatenated with the instruction and discretized robot-state tokens and encoded by an 18-layer Gemma 3 270M backbone with hidden dimension 640.

A separate 18-layer value expert with hidden dimension 256 updates a zero-initialized learnable [CLS] token. Through shared attention key-value representations, this token attends to the complete vision-language prefix while the prefix cannot attend back to it. A linear head maps the resulting state representation to the 201 value atoms:

$$h_t^{\text{CLS}} \in \mathbb{R}^{256} \longrightarrow \text{logits}_t = W_V h_t^{\text{CLS}} + b_V \in \mathbb{R}^{201}. \quad (21)$$

The vision encoder and Gemma backbone are initialized from pretrained weights. The value expert and linear head use their default random initialization, and the [CLS] token is initialized to zero. All components are optimized jointly.

N -step advantage labeling. Because the value targets are normalized to $[-1, 0]$, we express the transition rewards used for advantage estimation on the same scale:

$$\tilde{r}_t = \begin{cases} y_t - y_{t+1}, & t < T, \\ y_T, & t = T. \end{cases} \quad (22)$$

For a complete N -step horizon, the advantage at a valid action-chunk start is

$$A_t^{(N)} = \sum_{k=0}^{N-1} \tilde{r}_{t+k} + V(o_{t+N}, \ell) - V(o_t, \ell). \quad (23)$$

If fewer than N steps remain, rewards are accumulated through termination and the bootstrap value beyond the episode is set to zero. We convert the continuous estimate into a prompt-routing label,

$$I_t = \mathbb{I}[A_t^{(N)} \geq \epsilon], \quad (24)$$

where $N = 50$ matches the action horizon $H = 50$ and ϵ is the 70th percentile of the advantages over the aggregated dataset. The label is attached to the valid start frame t and therefore applies to the complete action chunk beginning at that frame.

Advantage-conditioned policy training. The binary label determines the condition appended to the task instruction:

$$c_t = \begin{cases} \text{Advantage: positive}, & I_t = 1, \\ \emptyset, & I_t = 0. \end{cases} \quad (25)$$

For a positive sample, the condition is independently dropped with probability $p_{\text{drop}} = 0.3$; a non-positive sample always uses the base prompt. Demonstration samples and any human-intervention samples included in the RECAP dataset are assigned the positive label. Advantage affects only this prompt routing: all retained policy samples use the same masked flow-matching action loss, with padded action positions and masked action dimensions excluded from the corresponding reductions. The value model is trained separately with the value-bin cross-entropy in Equation (20).

Action-space classifier-free guidance. At inference, we denoise a base prediction $\mathbf{a}_{\text{base}}$ and a positive-conditioned prediction $\mathbf{a}_{\text{pos}}$ from the same initial noise, then combine the completed action chunks once in action space:

$$\mathbf{a}_{\text{CFG}} = \mathbf{a}_{\text{base}} + \beta (\mathbf{a}_{\text{pos}} - \mathbf{a}_{\text{base}}), \quad \beta = 2.5. \quad (26)$$

This is a one-shot action-space approximation rather than guidance applied to the velocity field at every denoising step. Sharing the initial noise makes the difference between the two chunks reflect the advantage condition rather than independent sampling noise.

Training configuration. Table 8 lists the configuration used in the current experiments.

Table 8: RECAP value learning, advantage labeling, and policy-training hyperparameters.

Parameter	Setting
Value support $[V_{\min}, V_{\max}]$	$[-1, 0]$
Number of value atoms B	201
Value training steps / batch size	6,000 / 256
Value peak learning rate	5×10^{-5}
Value warmup / final learning rate	500 / 1×10^{-6}
Advantage horizon N / action horizon H	50 / 50
Advantage threshold ϵ	Aggregated-data 70th percentile
Condition-dropout rate p_{drop}	0.3
Policy training steps / batch size	30,000 / 128
Policy peak learning rate	1×10^{-5}
Policy warmup / minimum learning rate	10,000 / 5×10^{-7}
Inference guidance scale β	2.5

References

- [1] AgiBot-World-Contributors et al. AgiBot World Colosseo: A large-scale manipulation platform for scalable and intelligent embodied systems. *arXiv preprint arXiv:2503.06669*, 2025.
- [2] Kaiwen Bi et al. Motus: Motion priors improve vision-language-action models. *arXiv preprint*, 2025.
- [3] Kevin Black, Noah Brown, James Darpinian, Karan Dhariwal, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [5] Kevin Black, Manuel Y. Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, volume 38, pages 37596–37620, 2025.
- [6] Kevin Black, Allen Z. Ren, Michael Equi, and Sergey Levine. Training-time action conditioning for efficient real-time chunking. *arXiv preprint arXiv:2512.05964*, 2025.
- [7] Jisong Cai, Long Ling, Shiwei Chu, Zhongshan Liu, Jiayue Kang, Zhixuan Liang, Wenjie Xu, Yinan Mao, Weinan Zhang, Xiaokang Yang, et al. Aha-wam: Asynchronous horizon-adaptive world-action modeling with observation-guided context routing. *arXiv preprint arXiv:2606.09811*, 2026.
- [8] Zhenyu Cai et al. InternVLA-A1: Unifying understanding and action via latent action modeling. *arXiv preprint*, 2026.
- [9] Tianxing Chen, Yue Chen, Zixuan Li, Junyuan Tang, Kailun Su, Weijie Wan, Baijun Chen, Haoran Lu, Haowen Yan, Honghao Su, et al. RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies. *arXiv preprint arXiv:2607.04434*, 2026.
- [10] Zi-Han Ding and Ziwei Wang. Skillplug: Unsupervised skill mining for few-shot adaptation in robotic manipulation. *IEEE Robotics and Automation Letters*, 2026.

- [11] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, and Robin Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In *International Conference on Machine Learning*, 2024.
- [12] Jun Guo, Qiwei Li, Peiyan Li, Zilong Chen, Nan Sun, Yifei Su, Heyun Wang, Yuan Zhang, Xinghang Li, and Huaping Liu. Unified 4d world action modeling from video priors with asynchronous denoising. *arXiv preprint arXiv:2604.26694*, 2026.
- [13] Dongjie Huo, Haoyun Liu, Guoqing Liu, Dekang Qi, Zhiming Sun, Maoguo Gao, Jianxin He, Yandan Yang, Xinyuan Chang, Feng Xiong, et al. Abot-claw: A foundation for persistent, cooperative, and self-evolving robotic agents. *arXiv preprint arXiv:2604.10096*, 2026.
- [14] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [15] Tao Jiang, Tianyuan Yuan, Yicheng Liu, Chenhao Lu, Jianning Cui, Xiao Liu, Shuiqi Cheng, Jiyang Gao, Huazhe Xu, and Hang Zhao. Galaxea Open-World Dataset and G0 Dual-System VLA Model. *arXiv preprint arXiv:2509.00576*, 2025.
- [16] Tao Jiang, Tianyuan Yuan, Yicheng Liu, Chenhao Lu, Jianning Cui, Xiao Liu, Shuiqi Cheng, Jiyang Gao, Huazhe Xu, and Hang Zhao. Galaxea open-world dataset and g0 dual-system vla model. *arXiv preprint arXiv:2509.00576*, 2025.
- [17] Michael Kelly, Chelsea Sidrane, Katherine Driggs-Campbell, and Mykel J. Kochenderfer. HG-DAGger: Interactive imitation learning with human experts. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 8077–8083, 2019.
- [18] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024.
- [19] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. *arXiv:2304.02643*, 2023.
- [20] LejuRobotics. LET: Full-size humanoid robot real-world dataset. [Hugging Face dataset](#), 2025.
- [21] Fuhao Li, Wenxuan Song, Han Zhao, Jingbo Wang, Pengxiang Ding, Donglin Wang, Long Zeng, and Haoang Li. Spatial forcing: Implicit spatial representation alignment for vision-language-action model. In *International Conference on Learning Representations*, volume 2026, pages 132324–132345, 2026.
- [22] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Yang Zhaohui, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, et al. Simplevla-rl: Scaling vla training via reinforcement learning. In *International Conference on Learning Representations*, volume 2026, pages 125846–125869, 2026.
- [23] Hengtao Li, Pengxiang Ding, Runze Suo, Yihao Wang, Zirui Ge, Dongyuan Zang, Kexian Yu, Mingyang Sun, Hongyin Zhang, Donglin Wang, et al. Vla-rft: Vision-language-action reinforcement fine-tuning with verified rewards in world simulators. *arXiv preprint arXiv:2510.00406*, 2025.
- [24] Ruicheng Li et al. LingBot-VA: A vision-action model for robotic manipulation. *arXiv preprint*, 2026.
- [25] Ruiying Li, Yunlang Zhou, Yu Yao Zhu, Kylin Chen, Jingyuan Wang, Sukai Wang, Kongtao Hu, Minhui Yu, Bowen Jiang, Zhan Su, et al. Roboclaw: An agentic framework for scalable long-horizon robotic tasks. *arXiv preprint arXiv:2603.11558*, 2026.
- [26] Tianhong Li and Kaiming He. Back to basics: Let denoising generative models denoise. *arXiv preprint arXiv:2511.13720*, 2025.
- [27] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.
- [28] Yicheng Liu, Zibin Dong, Baijun Ye, Tianyuan Yuan, Tao Jiang, Anqi Yang, Shicheng Cao, Haonan Liu, Yue Sun, Zihan Guo, et al. G0. 5: One autoregressive stream for robot reasoning and action. *arXiv preprint arXiv:2608.11739*, 2026.
- [29] Zhiyu Luo et al. Being-H0.7: A vision-language-action model with hierarchical grounding. *arXiv preprint*, 2026.
- [30] Haoxiang Ma, Junhao Cai, Xiaoxu Xu, Hao Li, Yuyin Yang, Yang Tian, Jiafei Cao, Hongrui Zhu, Zherui Qiu, Yuqiang Yang, et al. Internvla-a1. 5: Unifying understanding, latent foresight, and action for compositional generalization. *arXiv preprint arXiv:2607.04988*, 2026.

- [31] Yao Mu, Tianxing Chen, Shijia Peng, Zanzin Chen, Zeyu Chen, Qiangyu Feng, Zhiqian Su, Weiliang Zhou, Guodong Ma, et al. RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.
- [32] NVIDIA, Johan Bjorck, et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [33] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. FAST: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- [34] Physical Intelligence Team, Shanghai AI Laboratory. InternVLA-A1.5: Unifying understanding, latent foresight, and action for compositional generalization. *arXiv preprint arXiv:2607.04988*, 2026.
- [35] Qwen Team. Qwen-VLA: Unifying vision-language-action modeling across tasks, environments, and robot embodiments. *arXiv preprint arXiv:2605.30280*, 2026.
- [36] Qwen Team. Qwen3.5: Towards native multimodal agents, 2026. <https://qwen.ai/blog?id=qwen3.5>.
- [37] RLinf Team. RECAP: Offline advantage-based policy optimization. RLinf Documentation, 2026. Accessed August 19, 2026.
- [38] StarVLA Community. Starvla: A lego-like codebase for vision-language-action model developing. *arXiv preprint arXiv:2604.05014*, 2026.
- [39] StarVLA Community. StarVLA: A modular framework for vision-language-action research, 2026. <https://github.com/StarVLA>.
- [40] Xiaomi Robotics Team, Jun Guo, Piaopiao Jin, Jason Li, Peiyan Li, Yingyan Li, Futeng Liu, Wanli Peng, Optimus Qin, Yifei Su, et al. Xiaomi-robotics-1: Scaling vision-language-action models with over 100k hours of real-world trajectories. *arXiv preprint arXiv:2607.15330*, 2026.
- [41] Yang Tian, Yuyin Yang, Yiman Xie, Zetao Cai, et al. InternData-A1: Pioneering high-fidelity synthetic data for pre-training generalist policy. *arXiv preprint arXiv:2511.16651*, 2025.
- [42] Songlin Wei, Hongyi Jing, Boqian Li, Zhenyu Zhao, Jiageng Mao, Zhenhao Ni, Sicheng He, Jie Liu, Xiawei Liu, Kaidi Kang, et al. ψ_0 : An open foundation model towards universal humanoid loco-manipulation. *arXiv preprint arXiv:2603.12263*, 2026.
- [43] Junjie Wu et al. LingBot-VLA: A vision-language-action model for robotic manipulation. *arXiv preprint*, 2026.
- [44] Kun Wu, Chengkai Hou, Jiaming Liu, Zhengping Che, Xiaozhu Ju, Zhuqin Yang, Meng Li, YINUO Zhao, Zhiyuan Xu, Guang Yang, et al. RoboMIND: Benchmark on multi-embodiment intelligence normative data for robot manipulation. *arXiv preprint arXiv:2412.13877*, 2024.
- [45] Shihan Wu et al. RoboCOIN: An open-sourced bimanual robotic data collection for integrated manipulation. *arXiv preprint arXiv:2511.17441*, 2025.
- [46] X-VLA Team. X-VLA: Soft-prompted transformer as a scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2504.20946*, 2026.
- [47] Heqing Yang, Yang Yi, Liyao Wang, Linqing Zhong, Donglin Yang, Ruipu Wu, Zitong Bai, Fengjiao Chen, Manyuan Zhang, Linjiang Huang, and Si Liu. EVA-Client: A unified data collection, inference, and deployment framework for embodied policies on real robots. *arXiv preprint arXiv:2607.02646*, 2026.
- [48] Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. Set-of-Mark prompting unleashes extraordinary visual grounding in GPT-4V. *arXiv preprint arXiv:2310.11441*, 2023.
- [49] Xingcheng Yang et al. ABot-M0: A vision-language-action model with action manifold learning for bimanual robotic manipulation. *arXiv preprint arXiv:2602.11236*, 2026.
- [50] Angen Ye, Boyuan Wang, Chaojun Ni, Guan Huang, Guosheng Zhao, Hao Li, Hengtao Li, Jie Li, Jindi Lv, Jingyu Liu, et al. Gigaworld-policy: An efficient action-centered world-action model. *arXiv preprint arXiv:2603.17240*, 2026.
- [51] Angen Ye, Zeyu Zhang, Boyuan Wang, Xiaofeng Wang, Dapeng Zhang, and Zheng Zhu. Vla-r1: Enhancing reasoning in vision-language-action models. *arXiv preprint arXiv:2510.01623*, 2025.
- [52] Chenghao Yin et al. Genie Sim 3.0: A high-fidelity comprehensive simulation platform for humanoid robot. *arXiv preprint arXiv:2601.02078*, 2026.

- [53] Checheng Yu, Chonghao Sima, Gangcheng Jiang, Hai Zhang, Haoguang Mai, Hongyang Li, Huijie Wang, Jin Chen, Kaiyang Wu, Li Chen, Lirui Zhao, Modi Shi, Ping Luo, Qingwen Bu, Shijia Peng, Tianyu Li, and Yibo Yuan. χ_0 : Resource-aware robust manipulation via taming distributional inconsistencies. *arXiv preprint arXiv:2602.09021*, 2026.
- [54] Ye Yuan et al. Fast-WAM: World-aware manipulation with fast action generation. *arXiv preprint*, 2026.
- [55] He Zhang, Lingzhu Xiang, Haitao Lin, Zeyu Huang, Minghui Wang, Dingyan Zhong, Yubo Dong, Yihao Wu, Yongming Rao, Dongsheng Zhang, et al. Hy-embodied-0.5-vla: From vision-language-action models to a real-world robot learning stack. *arXiv preprint arXiv:2606.14409*, 2026.
- [56] Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Chunyang Zhu, Jiaxing Qiu, Yuchen Yan, Jiayuan Liu, Wenhao Tang, et al. Harness vla: Steering frozen vlcs into reliable manipulation primitives via memory-guided agents. *arXiv preprint arXiv:2607.08448*, 2026.
- [57] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023.
- [58] Yi Zhou, Connelly Barnes, Jingwan Lu, Jimei Yang, and Hao Li. On the continuity of rotation representations in neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5745–5753, 2019.